Reproducing DFF-Adapter: Fine-Grained DINOv2 Tuning with Dual Supervision for Face Forgery Detection and Its Cross-Dataset Generalization

Sudhiksha Mattewada (B24CS027) and Moguluri Sai Siddhardha (B24DS018)

***Abstract*—Deepfake detectors that score highly on their training benchmark often fail on unseen manipulations. We re-implement DFF-Adapter, a parameter-efficient detector that inserts multi-head LoRA-expert adapters, with a shared head and task-specific heads, into the query, value and output projections of every transformer block of a frozen DINOv2 backbone, and trains authenticity detection jointly with forgery-type classification.**

Because of computational constraints, we use DINOv2-Small and DINOv2-Base instead of the paper's DINOv2-Large. Trained on FaceForensics++ (c23), our best configuration (Small, 30 epochs) reaches 99.08% FF++ validation video AUC (paper: 99.56%, test split) and 82.98%, 86.25% and 88.03% video AUC on Celeb-DF-v1, Celeb-DF-v2 and a 392-video DFDC subset, respectively (paper: 96.14%, 95.26% and 89.96%).

The gap to the paper is therefore 9–13 percentage points (pp) on Celeb-DF but under 2 pp on our DFDC subset, whose composition may differ from the paper's. A larger backbone did not improve our cross-dataset results: Base trails Small by 4.03 pp video AUC on DFDC. As an extension, label smoothing (0.1) on the forgery-type loss did not improve video-level AUC on any unseen set.

All results are single-seed. Checkpoint selection differs across evaluation datasets and experiments; see Section VIII.

***Index Terms*—Deepfake detection, DINOv2, LoRA, mixture of experts, parameter-efficient fine-tuning, cross-dataset generalization, label smoothing.**

I. Introduction

Synthetic face manipulation has become increasingly realistic, making automated and generalizable deepfake detection an important problem in computer vision. A major challenge is not simply achieving high accuracy on a known benchmark, but maintaining reliable performance when the manipulation method, compression, or identity distribution changes.

Recent work therefore adapts large pre-trained vision models for face forgery detection. DFF-Adapter addresses this problem using a frozen DINOv2 backbone and task-specific adapter branches. Unlike approaches that treat detection as generic real/fake classification, it jointly performs authenticity detection and forgery-type classification, allowing the authenticity branch to benefit from manipulation-specific features.

Project criterion. We follow the "reproduce paper results" criterion: we re-implemented the method from the paper's description and did not run the authors' implementation. The paper's reported numbers are therefore used as reference values for comparison. The project proposal recorded that no official code release could be found for arXiv:2511.12107v1 at the time of writing.

Contributions.

- A re-implementation of DFF-Adapter using frozen DINOv2-Small and DINOv2-Base backbones.
- Evaluation on FF++, Celeb-DF-v1, Celeb-DF-v2 and a DFDC subset, compared with the paper's reported video-level AUC and EER.
- Controlled comparisons of backbone size and training length, along with an extension using label smoothing on the forgery-type loss.
- An analysis of protocol differences, checkpoint selection and experimental limitations that affect reproducibility.

Team composition and contributions.

- **Member 1: Sudhiksha Mattewada (B24CS027).** DFF-Adapter Architecture and Experimental Analysis Lead. Responsibilities include adapter architecture, LoRA expert bank and aggregation, multi-head routing and top-3 selection, shared-head adapter and task fusion, dual-task training, loss design, hyperparameter tuning, ablations, label smoothing and routing/result analysis.
- **Member 2: Moguluri Sai Siddhardha (B24DS018).** Data, Backbone and Baseline Lead. Responsibilities include dataset acquisition and organization, leakage-safe splits, preprocessing and frame sampling, DINOv2 integration and freezing, baseline implementation, evaluation infrastructure and result verification.

II. Related Work

Deepfake detection datasets. FaceForensics++ provides real and manipulated facial videos generated using multiple manipulation methods and serves as our primary training dataset. Celeb-DF and the DeepFake Detection Challenge dataset are used to evaluate cross-dataset generalization.

Pre-trained vision models. DINOv2 learns general-purpose visual features through self-supervised training. The DFF-Adapter paper uses the DINOv2 model with registers, motivated by the register mechanism introduced by Darcet et al. We use the Small and Base DINOv2 checkpoints because of computational constraints.

Parameter-efficient fine-tuning. Low-Rank Adaptation (LoRA) introduces trainable low-rank updates while keeping the original model weights frozen. This reduces the number of trainable parameters compared with full fine-tuning. Mixture-of-experts approaches have also been explored for parameter-efficient face forgery detection, including MoE-FFD.

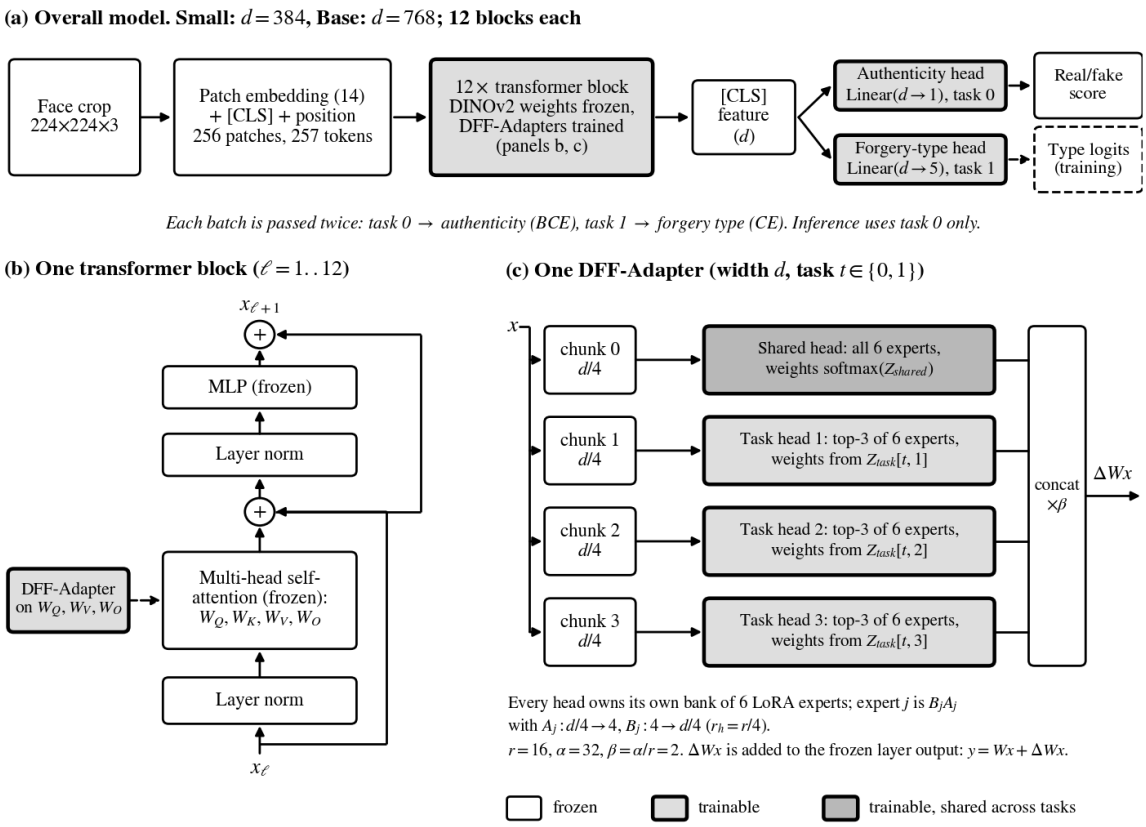


Fig. 1. Pipeline of the implemented DFF-Adapter approach. Raw videos are frame-sampled, faces are detected and cropped, and a frozen DINOv2 backbone augmented with shared and task-specific LoRA adapter heads is jointly supervised using authenticity and forgery-type losses.

DFF-Adapter. DFF-Adapter integrates low-rank adapters into every transformer block and jointly learns authenticity detection and forgery-type classification. Its Forgery-Aware Multi-Head Router selects the top-three LoRA experts for each task-specific head, while Shared-Enhanced Task Fusion allows the authenticity branch to benefit from features learned by the auxiliary forgery-type task.

Label smoothing. Label smoothing replaces hard one-hot targets with softened probability distributions. Its effects on model generalization and calibration have been studied by Müller et al. We investigate label smoothing as an extension to the original DFF-Adapter method by applying it only to the auxiliary forgery-type classification loss.

III. Proposed Methodology

A. Overview

Fig. 1 summarises the model. A 224×224 face crop is split into 14×14 patches (256 patches plus [CLS], 257 tokens) and encoded by a frozen DINOv2 backbone with 12 transformer blocks.

In every block, three linear layers are wrapped as $y = Wx + \Delta Wx$ with the frozen W and a trainable DFF-Adapter: the query, value and attention-output projections. A linear head on the [CLS] token produces the authenticity logit (task 0) or the five-way forgery-type logits (task 1). Only the adapters and the two linear heads are trained.

B. DFF-Adapter

Let $x \in \mathbb{R}^{L \times d}$ be the input of a wrapped layer and $H = 4$ the number of heads. The channels are split into H chunks $x^{(k)} \in \mathbb{R}^{L \times d/H}$.

Each head k owns a bank of $N = 6$ LoRA experts $(A_j^{(k)}, B_j^{(k)})$ with

$$A_j^{(k)} \in \mathbb{R}^{r_h \times d/H}, \qquad B_j^{(k)} \in \mathbb{R}^{d/H \times r_h},$$

and per-head rank $r_h = r/H = 4$, where $r = 16$.

Task heads ($k = 1, 2, 3$). For task $t \in \{0, 1\}$ (authenticity, forgery type), the routing probabilities are

$$g_{t,k} = \mathrm{softmax}(Z_{\mathrm{task}}[t, k]).$$

The top three experts $S_{t,k}$ are kept and their weights renormalised, giving

$$f_t^k = \beta \sum_{j \in S_{t,k}} \tilde{g}_{t,k}^{(j)} B_j^{(k)} A_j^{(k)} x^{(k)}, \tag{1}$$

where

$$\tilde{g}_{t,k}^{(j)} = \frac{g_{t,k}^{(j)}}{\sum_{\ell \in S_{t,k}} g_{t,k}^{(\ell)}}. \tag{2}$$

Shared head ($k = 0$). It mixes all experts with $\mathrm{softmax}(Z_{\mathrm{shared}})$ and is common to both tasks:

TABLE I
BACKBONES (PATCH SIZE 14, 224×224 INPUT) AND TRAINABLE PARAMETERS.

	Small	Base	Large
Blocks / hidden size d	12 / 384	12 / 768	24 / 1024
Native attention heads	6	12	16
Adapter head width $d/4$	96	192	256
Adapter parameters	665,064	1,328,616	3,541,968
Two linear heads	2,310	4,614	6,150
Trainable total	**667,374**	1,333,230	3,548,118

$$f_{\text{share}} = \beta \sum_{j=1}^{N} \text{softmax}(Z_{\text{shared}})_j B_j^{(0)} A_j^{(0)} x^{(0)}. \quad (3)$$

The head outputs are concatenated along the channel axis,

$$\Delta W x = \text{concat}(f_{\text{share}}, f_t^1, f_t^2, f_t^3),$$

and added to the frozen layer output, with

$$\beta = \frac{\alpha}{r} = \frac{32}{16} = 2.$$

The routing logits $Z_{\text{task}} \in \mathbb{R}^{2\times3\times6}$ and $Z_{\text{shared}} \in \mathbb{R}^6$ are learned parameters per adapter that do not depend on the input; they are initialised from $\mathcal{N}(0, 0.5^2)$, a choice of ours.

A is initialised uniformly and B at zero, so every adapter starts as the identity. The shared head receives gradients from both losses, which is how forgery-type information reaches the authenticity stream.

C. Backbones and trainable parameters

We use the frozen `facebook/dinov2-small` and `facebook/dinov2-base` checkpoints (no registers); the paper uses `dinov2-with-registers-large`.

One adapter has

$$HN \cdot 2r_h(d/H) + 2(H-1)N + N$$

parameters, and there are three adapters per block. Table I lists the settings and counts.

The Small count equals the value logged by our training script; the paper's reported 3.5M is consistent with our formula applied to the Large backbone (about 3.55M).

The Base and Large totals are computed from the closed-form expression, not logged.

D. Dual-supervision objective

Each batch is passed through the network twice, once per task, and one backward pass is made on

$$\mathcal{L} = \lambda_a \,\text{BCE}(s, y_a) + \lambda_t \,\text{CE}(\mathbf{z}, \tilde{\mathbf{y}}_t), \quad (4)$$

where s is the authenticity logit from task 0, $\mathbf{z}$ the type logits from task 1, $y_a \in \{0, 1\}$ the authenticity label, and $\tilde{\mathbf{y}}_t$ the target over $K = 5$ classes (original, Deepfakes, Face2Face, FaceSwap, NeuralTextures).

We use $\lambda_a = 10$ and $\lambda_t = 2$, as in the paper. Training runs in fp16 autocast with gradient scaling.

TABLE II
EVALUATION SETS.

Dataset	Videos	Frames	Note
Celeb-DF-v1	100	3,200	100-video list
Celeb-DF-v2	518	16,576	test list
DFDC subset	392	12,544	76 real / 316 fake

E. Extension: label-smoothed forgery-type supervision

For the extension,

$$\tilde{\mathbf{y}}_t = (1-\epsilon)\mathbf{e}_{y_t} + \frac{\epsilon}{K}\mathbf{1},$$

with $\epsilon = 0.1$ (the standard label-smoothing target, implemented with the `label_smoothing` argument of the cross-entropy loss).

It is applied only to the forgery-type loss; the authenticity BCE is unchanged. The baseline uses hard labels, $\epsilon = 0$.

The motivation is that manipulation types may share artifacts, so hard labels may over-penalise plausible confusions between them.

F. Inference and video-level scoring

Inference uses task 0 only, so the forgery-type head adds no test-time cost. The frame score is $\text{sigmoid}(s)$, the probability of "fake", and the video score is the mean of the frame scores over all frames of the video, matching the paper's protocol.

IV. EXPERIMENTAL SETUP

A. Data

Training and validation. FaceForensics++ (c23): 5,000 videos covering the original class and four manipulations. Faces are cropped with MTCNN to 224×224 and 32 frames per video are stored. The training split has 3,600 videos.

Training samples. In each epoch every training video contributes a fixed number of frames drawn at random, so one epoch is 3,600× frames-per-video samples.

Light augmentation is applied: horizontal flip ($p = 0.5$), contrast $\alpha \in [0.8, 1.2]$ with brightness shift in $[-20, 20]$ ($p = 0.5$), and JPEG re-compression with quality 60–95 ($p = 0.3$). Images are normalised with ImageNet statistics. Validation uses 8 evenly spaced frames per video without augmentation.

Unseen evaluation sets use all 32 crops of every video (Table II).

B. Training configuration

Table III lists the settings. The optimiser is Adam with L2 weight decay (not decoupled), with a constant learning rate and no scheduler.

The paper trains DINOv2-Large for 50 epochs with learning rate 2×10^{-4} and weight decay 10^{-5} on one RTX 4090; our values differ in backbone, epochs, learning rate and weight decay.

A first full run (Small, 12 epochs) took about 77 minutes on one T4 at roughly 97 samples/s; the data loader alone sustained about 1,760 samples/s, so training is GPU-bound.

TABLE III
TRAINING CONFIGURATION OF THE REPORTED RUNS.

Backbone	DINOv2-Small / Base (frozen)
Adapter	$r = 16$, $\alpha = 32$, 6 experts, 4 heads, top-3
Epochs	12 or 30 (45,000 steps at 30)
Frames / video (train, val)	10, 8
Batch size	24
Learning rate, weight decay	10^{-4}, 0.01
Loss weights λ_a, λ_t	10, 2
Seed	42 (single seed)
Hardware	Kaggle Tesla T4 GPU

TABLE IV
VIDEO-LEVEL AUC (%), HIGHER IS BETTER. "–": NOT EVALUATED; N/R: NOT REPORTED. LS: LABEL SMOOTHING 0.1. PAPER VALUES ARE TAKEN FROM THE ORIGINAL DFF-ADAPTER PUBLICATION.

Model	Ep.	FF++	CDF-v1	CDF-v2	DFDC
Small	12	99.08	81.20	85.02	–
Small	**30**	99.08	**82.98**	**86.25**	**88.03**
Small + LS	30	99.05	81.20	84.74	87.45
Base	12	99.08	82.43	81.51	–
Base	30	**99.16**	81.71	84.22	84.00
Paper (Large)	50	99.56	96.14	95.26	89.96

C. Metrics and checkpoint selection

The primary metric is video-level AUC, as in the paper. AUC is computed from ranks (Mann–Whitney) with ties counted as one half. EER is the error at the point where the false-positive and false-negative rates are equal, found by linear interpolation of the ROC curve. We also report frame-level AUC.

After every epoch the script scores the validation split; `best.pt` is the epoch with the highest validation video AUC and `last.pt` the latest state.

Our FF++ column is this validation AUC, while the paper reports AUC on the FF++ test split.

The Celeb-DF numbers of the baseline runs come from the final checkpoint, while the DFDC and label-smoothing evaluations use `best.pt` (Section VIII).

D. Run-time validation

Before the full runs, a smoke test (250 FF++ videos, 2 epochs, two experiments concurrently on two T4 GPUs) confirmed that training exits cleanly and writes `best.pt`, `last.pt` and `done.json`. Its metrics (validation video AUC 0.41) are meaningless at that scale and are not reported as results.

The provided unit tests also check the adapter against a literal implementation of Eqs. (1)–(3).

V. RESULTS AND ANALYSIS

A. Video-level AUC

Table IV and Fig. 2 compare our models with the paper's reported values.

In-domain. All our models reach 99.05–99.16% FF++ validation AUC. This is within 0.4–0.5 pp of the paper's 99.56% (test split), but the two numbers come from different splits. Our FF++ EER is about 3.2–3.6% versus the paper's 1.79%.

TABLE V
CROSS-DATASET EER (%), LOWER IS BETTER.

Model	Ep.	CDF-v1	CDF-v2	DFDC
Small	12	30.65	22.94	–
Small	**30**	**26.32**	**22.35**	**19.74**
Small + LS	30	26.32	25.00	19.94
Base	12	25.81	26.18	–
Base	30	28.95	23.82	23.68
Paper (Large)	50	n/r	12.65	17.96

TABLE VI
DFDC SUBSET (392 VIDEOS, 12,544 FRAMES), `BEST.PT` CHECKPOINTS.

Metric	Small	Base	Better
Video AUC (%)	**88.03**	84.00	Small
EER (%)	**19.74**	23.68	Small
Frame AUC (%)	**79.99**	78.10	Small

Cross-dataset. Our best model (Small, 30 epochs) is 13.16 pp below the paper on Celeb-DF-v1 and 9.01 pp below on Celeb-DF-v2, but only 1.93 pp below on our DFDC subset (Fig. 3).

Our Base model does not outperform Small on either Celeb-DF evaluation set at 30 epochs, and it is 4.03 pp lower on DFDC. The DFDC comparison is weaker evidence: our subset (392 videos) may not be the paper's DFDC set.

B. Equal Error Rate

Table V mirrors the AUC picture. Small at 30 epochs has the lowest EER among our models on Celeb-DF-v2 (22.35% versus 12.65% for the paper) and on DFDC (19.74% versus 17.96%).

On Celeb-DF-v1 the 12-epoch Base model has a slightly lower EER (25.81%) than Small-30 (26.32%) although its AUC is lower (82.43% versus 82.98%). AUC and EER summarise different parts of the ROC curve and need not agree.

C. DFDC subset

Small is ahead on all three metrics (Table VI): $+4.03$ pp video AUC and -3.94 pp EER.

Video-level AUC exceeds frame-level AUC by 8.04 pp (Small) and 5.90 pp (Base), consistent with averaging damping noisy single-frame predictions.

Statistical uncertainty. The subset is small and imbalanced (76 real, 316 fake). The Hanley–McNeil approximation gives a standard error of about 1.75 pp for an AUC of 88%, i.e. a 95% interval of roughly ± 3.4 pp.

Small and Base are evaluated on the same videos, so their difference is paired and less uncertain than for independent samples, but the 4.03 pp gap and the 1.93 pp gap to the paper are both within or near this interval.

Differences of 1–2 pp, which dominate the Celeb-DF comparisons (only 100 videos in v1), lie within sampling noise.

VI. ABLATION STUDY

All ablations change one factor relative to a stated reference; deltas are in percentage points.

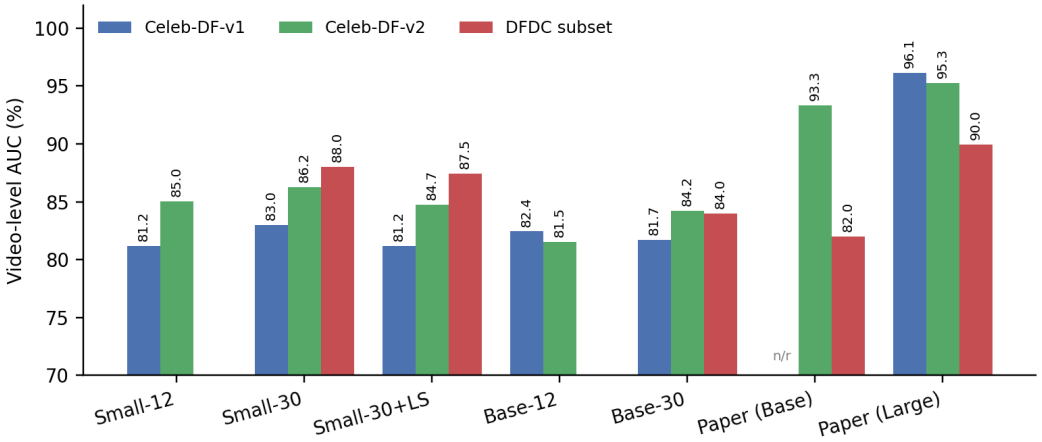


Fig. 2. Cross-dataset video AUC of all configurations against the paper (n/r: not reported).

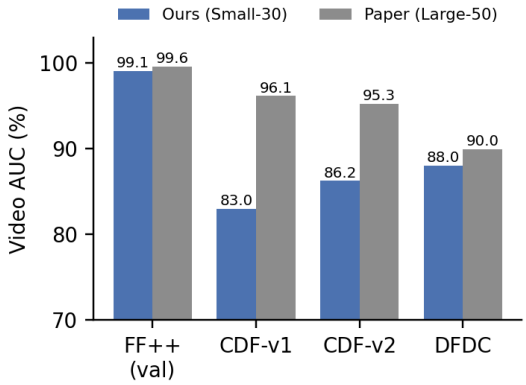


Fig. 3. In-domain (FF++) versus unseen-dataset video AUC for Small, 30 epochs, and the paper (Large).

TABLE VII
EFFECT OF 30 VERSUS 12 EPOCHS.

Backbone	ΔAUC v1	ΔEER v1	ΔAUC v2	ΔEER v2
Small	+1.78	−4.33	+1.23	−0.59
Base	−0.72	+3.14	+2.71	−2.36

A. Training length (12 → 30 epochs)

Longer training helps Small on both Celeb-DF sets, but for Base the effect depends on the target dataset (Table VII). Training duration alone is not a reliable predictor of generalization. DFDC is not available for the 12-epoch models.

B. Backbone size (Small versus Base)

At 30 epochs, Base minus Small is −1.27 pp on Celeb-DF-v1, −2.03 pp on Celeb-DF-v2 and −4.03 pp on DFDC video AUC, even though Base has the higher FF++ validation AUC (99.16% versus 99.08%).

At 12 epochs Base is +1.23 pp on v1 but −3.51 pp on v2. These results show that the effect of backbone size depends on the evaluation dataset and training duration in our experiments.

TABLE VIII
SMALL, 30 EPOCHS: LABEL SMOOTHING 0.1 MINUS BASELINE.

Dataset	Δ video AUC	Δ EER	Δ frame AUC
Celeb-DF-v1	−1.78	0.00	n/a
Celeb-DF-v2	−1.51	+2.65	n/a
DFDC subset	−0.58	+0.20	+0.91

C. Label smoothing on the forgery-type loss

The label-smoothed run (45,000 steps, best FF++ validation AUC 99.05%) did not improve video AUC on any unseen set (Table VIII). DFDC frame-level AUC rose by 0.91 pp, so frame- and video-level results are not uniformly aligned.

This is a single-seed result and does not show that label smoothing is generally ineffective. The baseline Celeb-DF numbers are final-checkpoint results whereas the label-smoothed evaluation used `best.pt`; this confound is disclosed in Section VIII.

D. Auxiliary supervision and adapter design

The current experiments do not include our own controlled ablations of the auxiliary forgery-type loss, plain LoRA adapters, or a linear-probe baseline. Therefore, we do not report numerical results for these configurations. Such experiments would be needed to isolate the contribution of dual supervision and the proposed adapter design in our implementation.

The original DFF-Adapter paper reports ablation experiments for its auxiliary supervision and adapter components. Those results are not treated as reproduced results in this report because we did not independently run the corresponding configurations. Our missing local ablations are therefore a limitation of this reproduction and are included in the future-work plan.

VII. DISCUSSION

Source-domain accuracy is not robustness. With about 99% FF++ validation AUC, Small-30 loses 16.10 pp on Celeb-DF-v1, 12.8 pp on Celeb-DF-v2 and 11.05 pp on DFDC.

A detector can separate real from fake very well on data like its training set and still struggle when the source, manipulation pipeline, compression or identity distribution changes. Which factor dominates cannot be determined from these experiments, and the FF++ and DFDC numbers may not belong to the same checkpoint.

Where the gap to the paper is. The shortfall is concentrated on Celeb-DF (9–13 pp), while on the DFDC subset we are within 2 pp of the paper's Large model.

Candidate causes of the Celeb-DF gap are:

1) Backbone capacity (Small/Base versus Large).
2) The training recipe (learning rate, weight decay, epochs).
3) Data and protocol differences (frames per video, splits, cropping, and the Celeb-DF lists).
4) Implementation details.

Several factors differ at once, so the comparison cannot attribute the gap to any one of them. It does not by itself show that the paper is irreproducible or that our implementation is wrong.

The parameter-count agreement with the paper's 3.5M budget and the unit tests against the literal equations support the adapter itself, but not the training recipe.

Bigger is not automatically better. In our runs Base wins on FF++ validation but loses to Small on DFDC on every metric. Plausible explanations (different learned representations, optimisation dynamics, interaction with the target data) remain hypotheses that need controlled runs. They also leave the paper's Base-versus-Large trend untested, since we have no Large run.

Longer training and overfitting. The mixed effect of 12 versus 30 epochs is consistent with overfitting to the source dataset, but it does not establish it.

Label smoothing. The data do not support a benefit at $\epsilon = 0.1$ in this run. The paper itself notes that artifacts of different manipulation methods may overlap, which could make the auxiliary task noisy. Smoothing was meant to soften that, but we did not measure type-classification quality, so we cannot say whether it did.

VIII. Limitations

1) **Backbone mismatch.** The paper uses DINOv2-Large with registers; we use plain Small and Base, so the comparison does not isolate implementation quality.
2) **Training mismatch.** Epochs, learning rate, weight decay, frames per video and hardware differ from the paper, and the paper follows DeepFakeBench defaults.
3) **Evaluation protocol.** Our manifests (100, 518 and 392 videos) have not been verified to match the paper's benchmark splits. The DFDC subset is a subset, so its comparison with the paper's DFDC number is indicative only.
4) **Validation versus test.** Our FF++ column is validation AUC, whereas the paper reports AUC on the FF++ test split.
5) **Checkpoint inconsistency.** FF++ numbers are best-validation checkpoints, Celeb-DF numbers of baseline runs are final checkpoints, and DFDC and label-smoothing results use `best.pt`. The training durations behind each `best.pt` should be confirmed before treating Table IV as a controlled comparison.
6) **Single seed.** No variance across seeds is available, so small differences cannot be separated from noise.
7) **No causal evidence.** The experiments show what changes, not why.
8) **Class imbalance and small sample size.** The DFDC subset has 76 real and 316 fake videos; Celeb-DF-v1 has only 100 videos. No confidence intervals beyond the approximation above were computed.
9) **Missing ablations.** Our own no-type-loss, plain-LoRA and linear-probe baselines are not reported (Section VI-D).

IX. Conclusion and Future Work

We re-implemented DFF-Adapter on smaller DINOv2 backbones. In-domain FF++ AUC (99.08% validation) is within about 0.5 pp of the paper's test AUC, but cross-dataset AUC on Celeb-DF is 9–13 pp lower, so the paper's Celeb-DF generalization is not reproduced in our setting.

On a 392-video DFDC subset we are within 2 pp of the paper's Large model (88.03% versus 89.96%), with the caveat that the subsets may differ.

Among our configurations, Small trained for 30 epochs is the strongest cross-dataset candidate. A larger backbone and label smoothing on the type loss did not help.

The most defensible conclusion is that source-dataset performance alone is insufficient to judge a deepfake detector.

Future work.

1) Evaluate one selected checkpoint per model on FF++ test and all unseen sets with a single scoring procedure.
2) Verify splits, cropping and the DFDC subset against DeepFakeBench.
3) Run at least three seeds for the best configuration.
4) Run the paper's ablations (no type loss, plain LoRA, linear probe) with our code and ablate label smoothing one factor at a time.
5) Evaluate the 12-epoch models on DFDC.
6) Try the paper's learning rate, weight decay and Large backbone if compute permits.

References

1) Zhang, T., Yu, P., Xia, Z., Dai, L., Zhou, X., and Gao, H., "Fine-Grained DINO Tuning with Dual Supervision for Face Forgery Detection," Proceedings of the AAAI Conference on Artificial Intelligence, vol. 40, no. 15, pp. 12780–12788, 2026.
2) Oquab, M., Darcet, T., Moutakanni, T., et al., "DINOv2: Learning Robust Visual Features without Supervision," Transactions on Machine Learning Research, 2024.
3) Darcet, T., Oquab, M., Mairal, J., and Bojanowski, P., "Vision Transformers Need Registers," International Conference on Learning Representations, 2024.
4) Hu, E. J., Shen, Y., Wallis, P., et al., "LoRA: Low-Rank Adaptation of Large Language Models," International Conference on Learning Representations, 2022.

5) Kong, C., Luo, A., Bao, P., et al., "MoE-FFD: Mixture of Experts for Generalized and Parameter-Efficient Face Forgery Detection," arXiv preprint arXiv:2404.08452, 2024.
6) Rössler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., and Nießner, M., "FaceForensics++: Learning to Detect Manipulated Facial Images," Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019.
7) Li, Y., Yang, X., Sun, P., Qi, H., and Lyu, S., "Celeb-DF: A Large-Scale Challenging Dataset for DeepFake Forensics," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020.
8) Dolhansky, B., Bitton, J., Pflaum, B., et al., "The DeepFake Detection Challenge (DFDC) Dataset," arXiv preprint arXiv:2006.07397, 2020.
9) Zhang, K., Zhang, Z., Li, Z., and Qiao, Y., "Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks," IEEE Signal Processing Letters, vol. 23, no. 10, pp. 1499–1503, 2016.
10) Müller, R., Kornblith, S., and Hinton, G., "When Does Label Smoothing Help?" Advances in Neural Information Processing Systems, vol. 32, 2019.
11) Hanley, J. A., and McNeil, B. J., "The Meaning and Use of the Area under a Receiver Operating Characteristic (ROC) Curve," Radiology, vol. 143, no. 1, pp. 29–36, 1982.

APPENDIX

Platforms. Training and evaluation ran as Kaggle notebooks on Tesla T4 GPUs (two GPUs used concurrently for two experiments); data were stored as private Kaggle datasets.

Software: PyTorch, Hugging Face Transformers (`Dinov2Model`), OpenCV for image loading and augmentation, and MTCNN for face cropping.

Reliability measures. The training script writes checkpoints atomically (temporary file, then rename) every 500 steps and at each epoch end, resumes mid-epoch from `last.pt` with a seed-and-epoch dependent sampler, stops cleanly after a maximum wall-clock time, and the runner evaluates a run only if `done.json` exists.

These were used so that long runs under Kaggle's commit mode could be monitored and recovered.

Use of AI assistance. Generative AI tools, primarily ChatGPT and Claude, were used as supporting tools during the development and documentation of this project. Their assistance was limited to technical guidance, code-related support, debugging, and report preparation. The major areas in which these tools were used are listed below:

1) **Understanding the research paper:** Assistance in interpreting the DFF-Adapter architecture, dual-supervision mechanism, LoRA-based adaptation, and experimental methodology.
2) **Implementation guidance:** Help with understanding implementation requirements, structuring the codebase, and translating the methods described in the research paper into an executable implementation.
3) **Code review and debugging:** Reviewing implementation logic, identifying potential errors, resolving runtime issues, and suggesting improvements to training, evaluation, and data-processing scripts.
4) **Experimental design:** Assistance in planning smoke tests, checking training configurations, structuring baseline and ablation experiments, and understanding appropriate evaluation metrics.
5) **Result interpretation:** Assistance in interpreting experimental outputs, comparing results across configurations and datasets, and identifying potential limitations and inconsistencies.
6) **Documentation and report preparation:** Assistance in organizing the report, improving technical explanations, preparing tables, and refining the presentation of methodology and experimental findings.

All implementation decisions, experiment execution, dataset preparation, result verification, and final conclusions were reviewed and validated by the project members. AI-generated suggestions were not treated as authoritative without checking them against the implementation, experimental outputs, and research paper. The project members retain responsibility for the correctness of the submitted work.